

Advanced Software Development

Johan Bontes (johan@digitsolutions.nl)

CSC3003S, 2023

Q & A

In some lectures, you skip slides [...]

Does this mean these will not be examined?

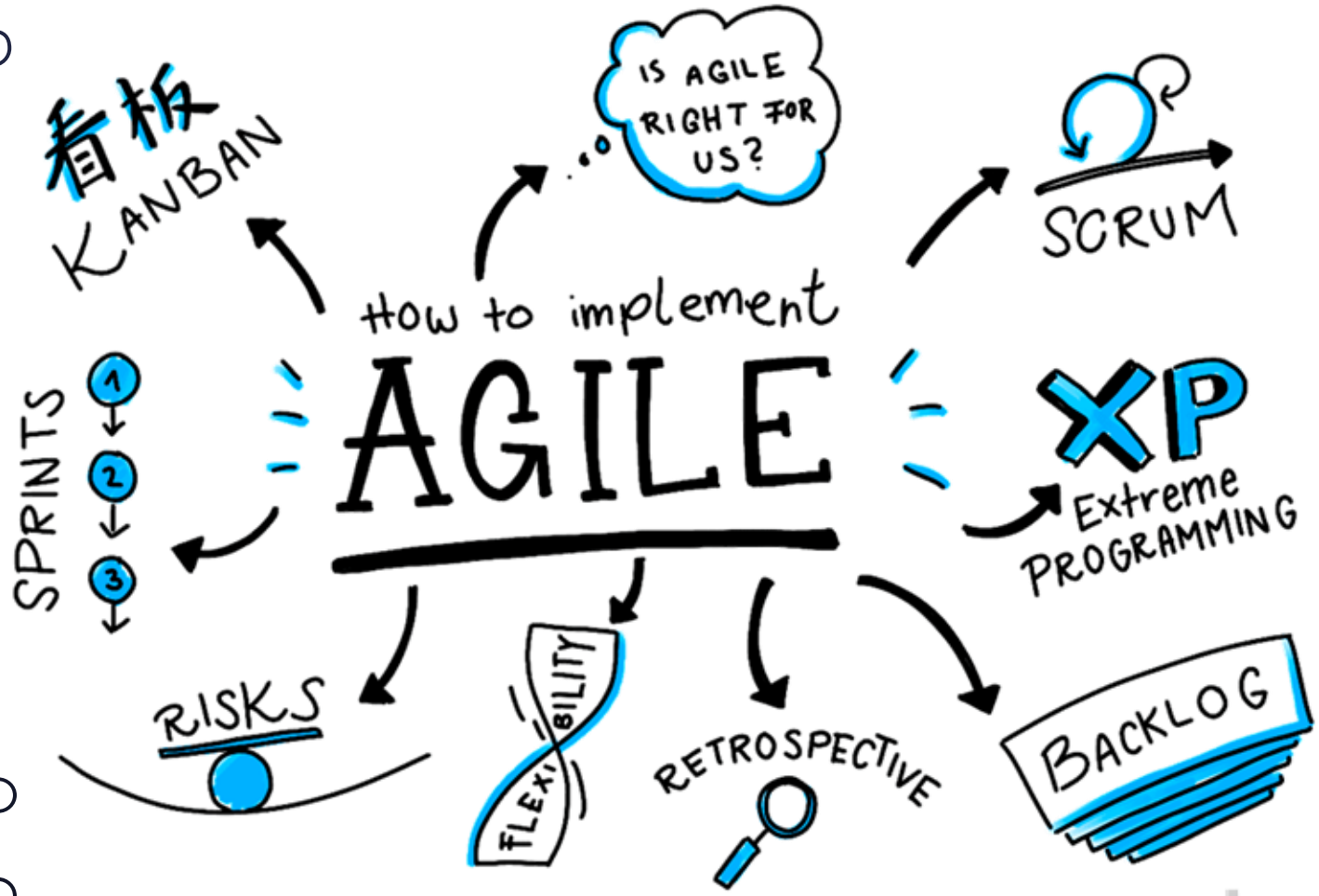
No, all material offered is examinable unless otherwise specified.

Purple subjects => not examinable

All other colours => examinable

Blue terms: high likelihood

Exam mostly about understanding, do not memorise lists



CSC3003S ASD03 Agile

A. Overview of Agile principles

B. Bare requirements

C. Fixed timescale

D. Working product at all times

E. Pair programming

The Agile manifesto

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

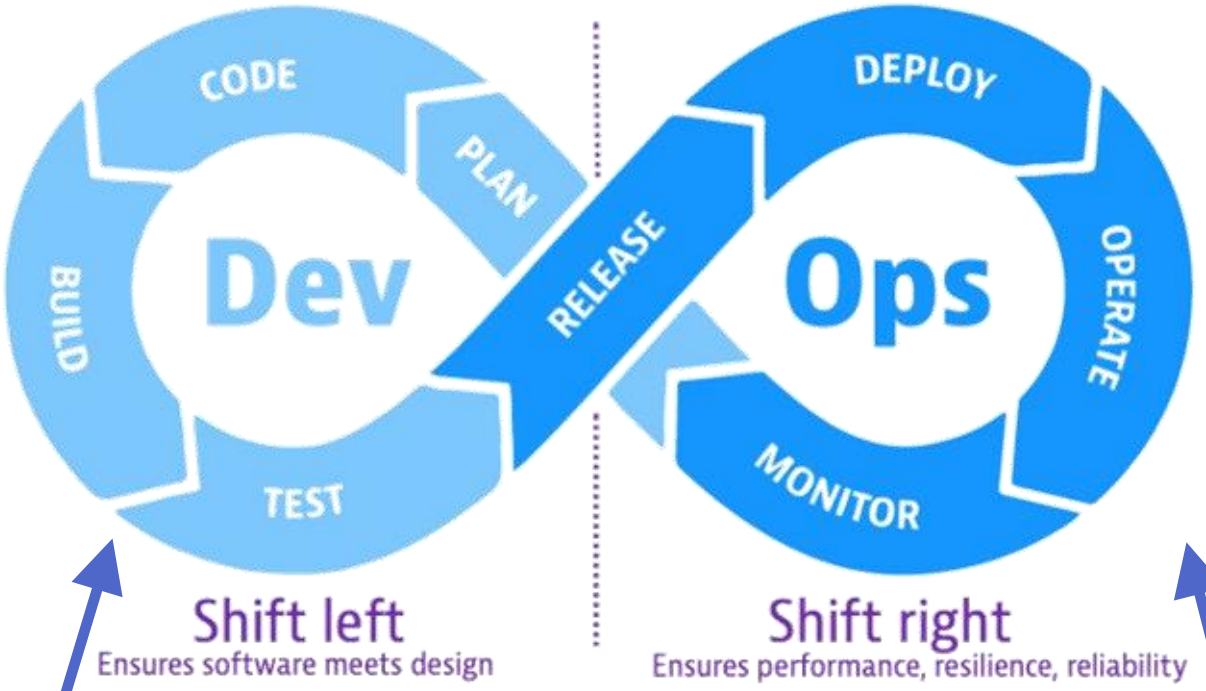
Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on
the right, we value the items on the left more.

Kent Beck - Mike Beedle - Arie van Bennekum - Alistair Cockburn - Ward Cunningham - Martin Fowler - James Grenning - Jim Highsmith - Andrew Hunt - Ron Jeffries - Jon Kern - Brian Marick - Robert C. Martin - Steve Mellor - Ken Schwaber - Jeff Sutherland - Dave Thomas

The Agile manifesto 15 years later

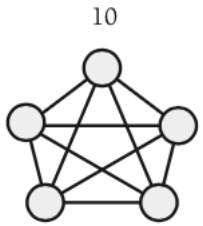
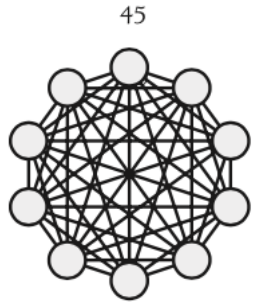
2015



2001

processes and tools	Individuals and interactions
comprehensive documentation	Working software
contract negotiation	Customer collaboration
following a plan	Responding to change

The Agile manifesto 15 years later

	processes and tools	Individuals and interactions
Communication	 easy	 hard
Division of labour	clear	less so
Task switching	few distractions	many
Team structure	hierarchy	democracy

The Agile manifesto 15 years later

Comprehensive documentation

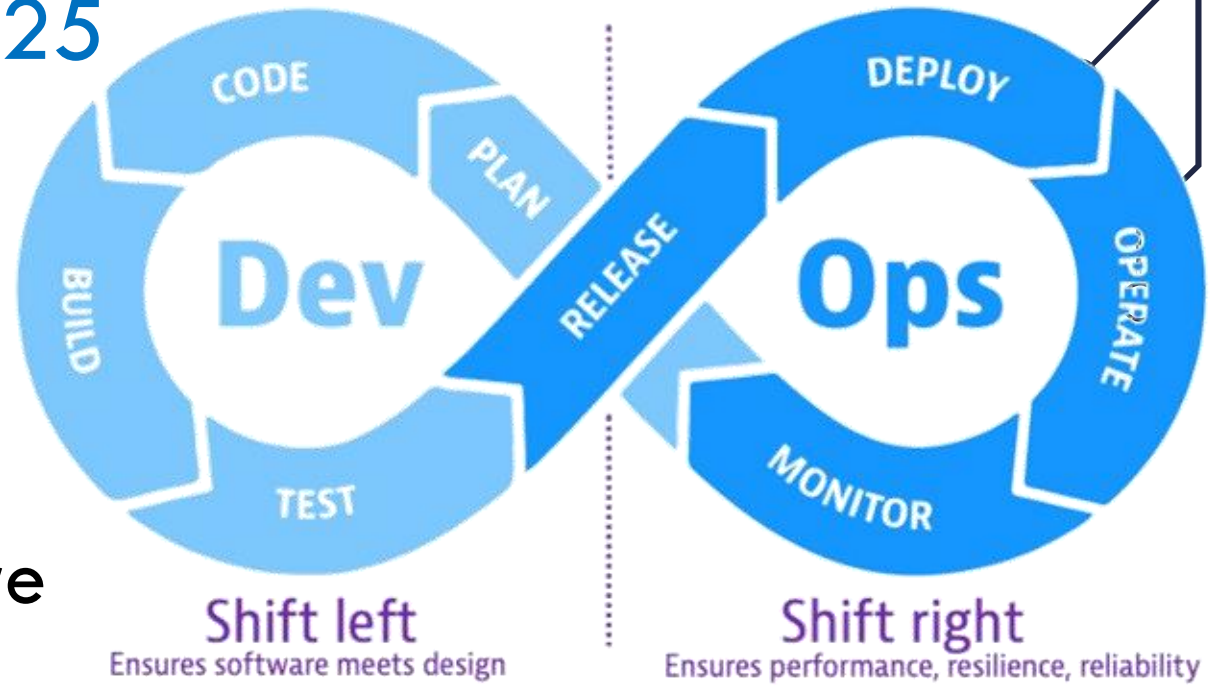
Working software

Contract negotiation

Customer collaboration

	Waterfall	Agile
Communication	easy	hard
Division of labour	clear	not clear
Works well for	known problems	novel development
final = plan	✓	✗

The reality of business in 2025



Top 3 problems are security × 3

Must be addressed in architecture

- more planning needed
- must follow defined secure patterns (mem/encrypt/auth)
- security cannot be 'bolted on' in the final iterations
- hard to test for
- users do not understand security
- and don't want it … because ↑security = ↓usability
- risk: insecure software = high probability + high cost = avoid

12 Agile Principles

@OlgaHeismann

Satisfy the customer through early and continuous delivery of valuable software.





Welcome changing requirements, even late in development.



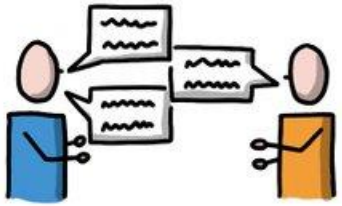
Deliver working software frequently.

Business people and developers must work together.



Build projects around motivated individuals. Give them the support they need. Trust them.





The most efficient and effective method of conveing information is face-to-face conversation.

Working software is the primary measure of progress.

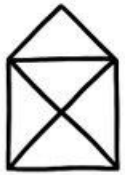




The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

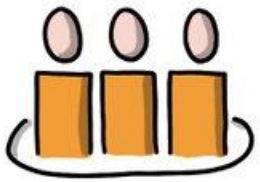
Continuous attention to technical excellence and good design.



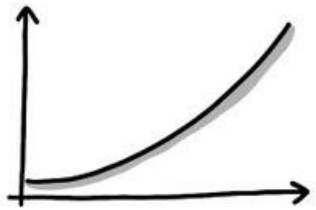


Simplicity—the art of maximizing the amount of work not done—is essential.

The best architectures, requirements, and designs emerge from self-organizing teams.

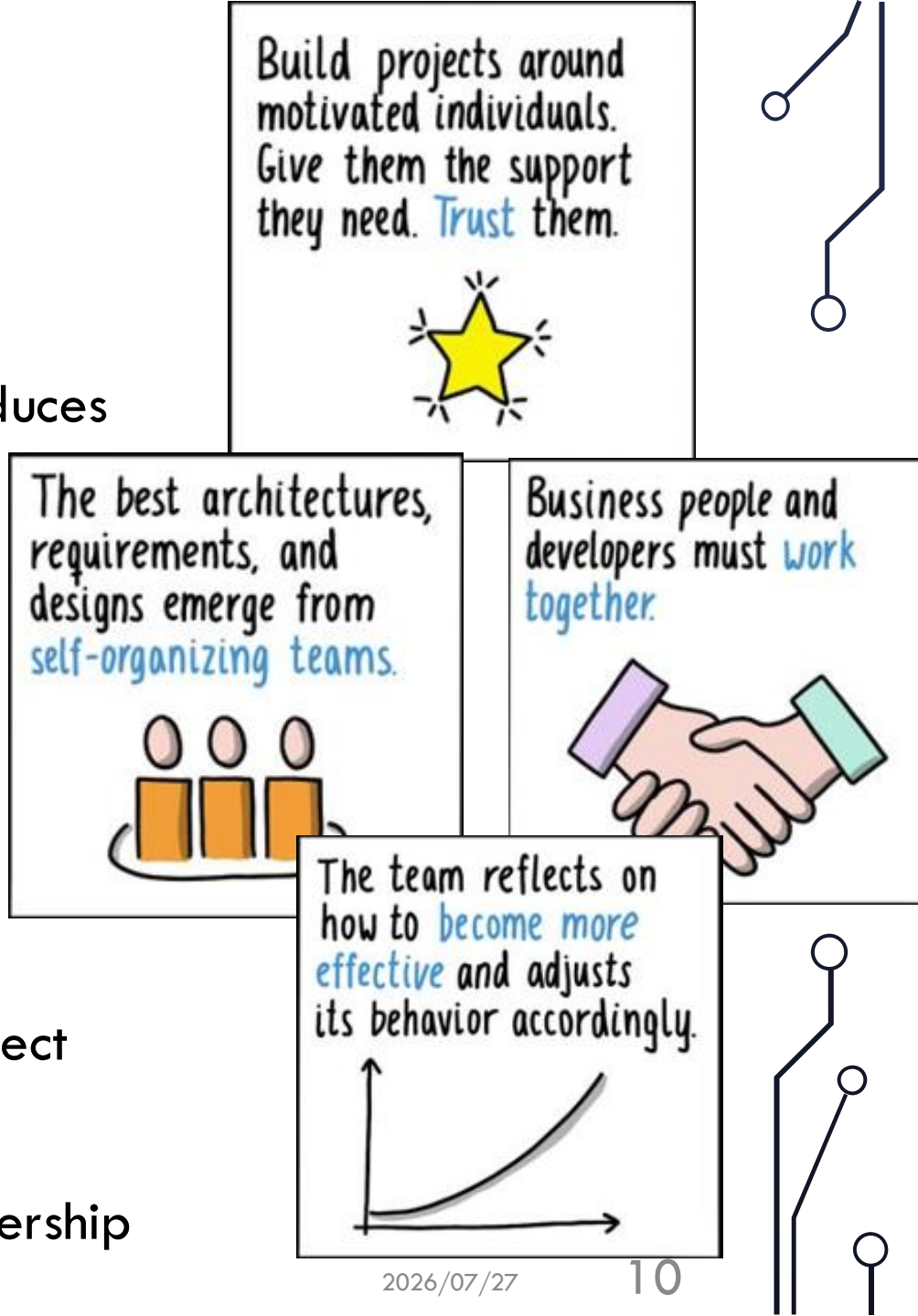


The team reflects on how to become more effective and adjusts its behavior accordingly.



Teams Must be Empowered

- The project team must have ***sole responsibility*** to deliver the product.
- Any interference with the project team is *disruptive* and reduces their motivation to deliver.
- The team must together
  - **establish and clarify the requirements,**
  - prioritise them together,
  - agree to the tasks required to deliver them,
  - estimate the effort involved.
- It ensures the **buy-in and commitment** from the entire project team from the outset
- When challenges arise, the team feels a real sense of ownership





CSC3003S ASD03 Agile

A. Overview of Agile Principles

B. Bare Requirements

C. Fixed Timescale

D. Working Product at All Times

E. Pair Programming

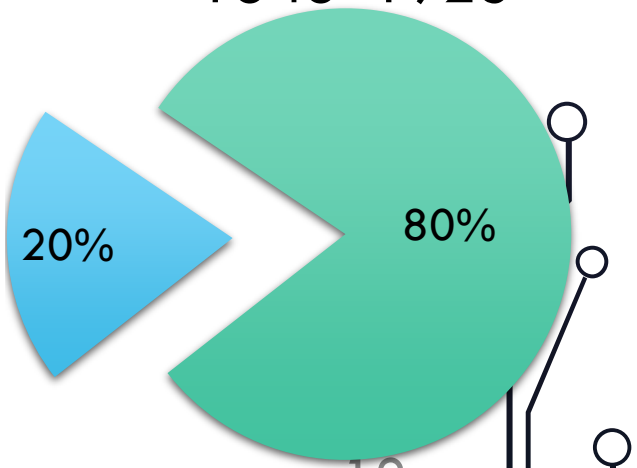
Pareto's Law – 80/20 rule

Note that the 80/20 division is arbitrary. It could just as well be 75/35, or most/minority

- *Typically,* ~80% of your results may come from only ~20% of your efforts!
  - try to apply the 80/20 rule and focus on the important 20% of effort that gets best results.
- Problem: no-one knows which 20% will deliver 80% of the results.



Vilfredo Pareto
1848–1923



Agile requirements are barely sufficient



- Capture requirements at a *high level* and on a *piecemeal* basis
  - just-in-time for each feature to be developed.
  - barely sufficient
    - The minimum to enable development and testing
    - minimise time spent on anything not part of product.
- Understand enough to determine the scope and for high level budgetary estimates.
- Captured in collaborative workshops so that all team members understand the requirements.
  - allows everyone to contribute, challenge and understand what's needed and why.

Agile requirements are barely sufficient

Risk assessment needed

- Capture requirements at a *high level* and on a *piecemeal* basis
  - just-in-time for each feature to be developed.
  - barely sufficient
    - The minimum to enable development and testing
    - minimise time spent on anything not part of product.
- Understand enough to determine the scope and for high level budgetary estimates.
- Captured in collaborative workshops so that all team members understand the requirements.
  - allows everyone to contribute, challenge and understand what's needed and why.

Only for low prob/low cost risks

Only for feature not well understood

Must also understand interaction and security aspects

This is an extreme form of democracy, aka anarchy

disregards (in)formal power

User Stories

- Most Agile teams represent each requirement as a user story.
  - similar to Use Cases but lightweight and simpler;
  - a simple statement about what a user wants to do with a feature;
  - should be written in business language understandable to all.



Form of user stories

As a [**user role**],

I want to [**tasks**],

So I can [**goal**].

- Should focus on the who, what, and why of a feature, ***not how***.
- On a job listing site, two high-level User Stories might be:
  - As a **job seeker**, I want to **search for a job**, so I can **advance my career**.
  - As a **recruiter**, I want to **post a job vacancy**, so I can **find a new team member**.

Capture user stories at the start

- At the start of a project, capture an initial list of User Stories up-front
  - **useful for estimating and planning.**
- **Defer capturing the details** until the story is prioritised and due to be developed.
- Users often tell stories
  - about the failings of their current system.
  - how they see things working better in future.
  - **Capture these stories as User Stories.** As they are told.
- In traditional development projects, these stories are captured in a lengthy analysis process and available in a lengthy document;
  - Not user friendly.

First rule of usability? Don't listen to users...

"*To design the best UX, pay attention to what users do, not what they say. Self-reported claims are unreliable, as are user speculations about future behavior. Users do not know what they want.*" – Jakob Nielsen

"*"a lot of times, people don't know what they want until you show it to them."* – Steve Jobs

Jobs meant to say: "users know what they (don't) want, but do not know what they need, until ...".

... you show it to them

Recording user stories

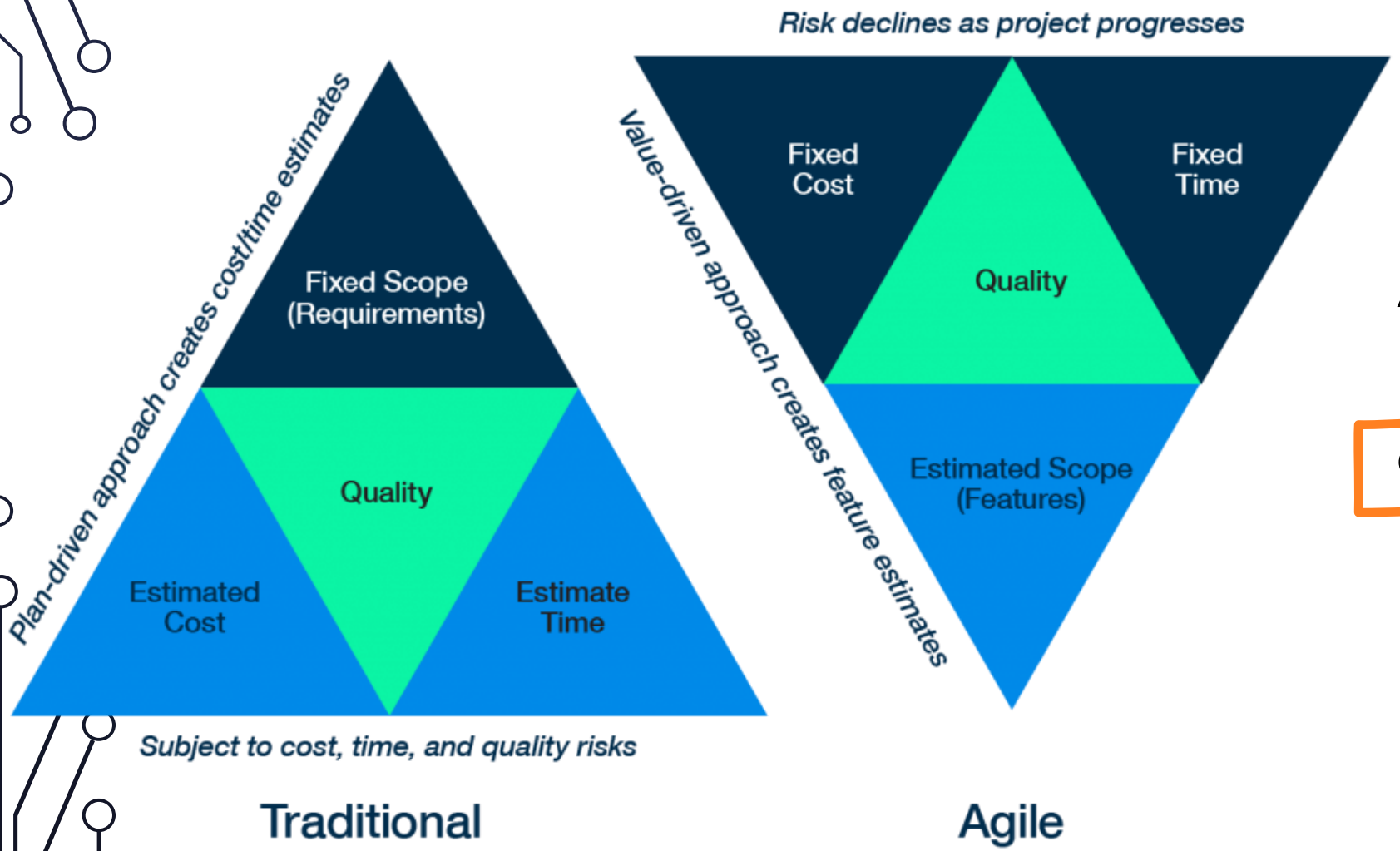
- Written on postcard size cards in 3 parts:
  - Heading: name/description, reference numbers, estimated size, etc.
  - Conversation *(on the front)*
    - background [who, why] + what system is meant to do
      - a **sketch or diagram** of the feature,
      - notes about how it should function.
  - Acceptance Criteria *(on the back)*
    - test cases to help identify scenarios that users, developers and/or analysts may not have thought of.
- Writing User Stories on a card forces requirements into small, manageable pieces of functionality
  - single features.

Agile requirements are barely sufficient

- Cards can be supported by documentation, but keep it to the bare minimum to allow a feature to be developed, and always in very small units.
- Requirements should be broken into tasks of ≤16 hours. Preferably 8 hours, so progress can be measured daily.
- All items are *deliverables* **not** activities or tasks.
  - You can see a deliverable to judge its quality and completeness.
  - A task you cannot see.

Incremental design

- As *opposed* to Fred Brooks - Waterfall
  Agile does *not* follow a top-down design method.
  - *Top-down design says: time in design is worth it if it saves cost in reworking the design many times.*
- Agile design is always the same size as the system.
  - "you can't possibly anticipate the problems and alternatives that will arise once you start coding".
    - = Unknown unknowns => not true for most problems
- If a new need comes along that requires major changes, then that is the trade-off for the flexibility it allows.
  - … perhaps this feature was not known at the beginning
  - or it might have gone away if we knew of it at the start!!



CSC3003S ASD08 Agile 1

A. Overview of Agile principles

B. Bare requirements

C. Fixed timescale

D. Working product at all times

E. Pair programming

Time Waits for No-one I

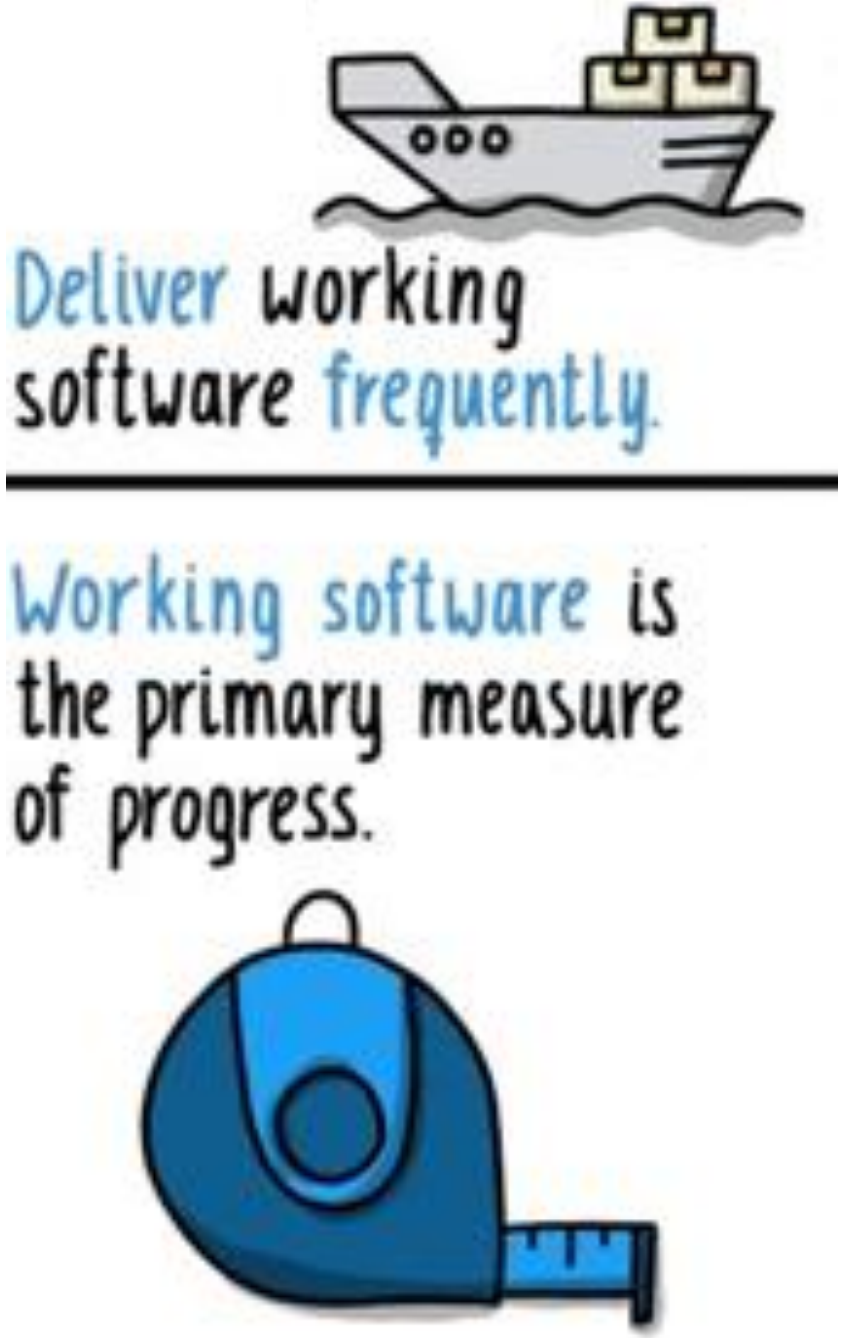
- In Agile Development, requirements evolve, but *timescales are fixed.*
- Contrast to traditional development:
  - capture all known requirements
  - changes are subject to change control.
  - users are told: it's more expensive to change requirements during or after the software is built.
    - Everything they can think of,
      everything they ever dreamed of must be included

Time Waits for No-one II

- Normally
  - Users may use only 20% of the product
  - Many projects start bloated.
    - No-one is sure at the outset which 20% they will use.
  - Cannot think of everything, things change, and things are understood differently.
- Agile Development **assumes changing requirements**
  - however much analysis and design you do, you *cannot really know* what you want until you see and use the software.
  - in the time spent analysing and reviewing requirements and designing a solution, external conditions could change

Fixed Budget

- What does business expect from development teams?
  - deliver the agreed business requirements
  - on time and within budget
  - to an acceptable quality.
    - In Agile Development
      - **scope** is variable,
        not the cost and timescale.
- For this to work, must start with **core, highest priority** features
  - delivered in the earliest iterations
- Result:
  - fixed budget,
  - fewer resources needed,
  - fixed launch date.



CSC3003S ASD03
AGILE

A. Overview of Agile principles

B. Bare requirements

C. Fixed timescale

D. Working product at all times

E. Pair programming

Agile Development Cycle

- The cycle is **Analyse**, **Develop**, **Test**, **repeat**
  - doing each step for each feature, one feature at a time.
- Advantages of this approach include:
  - **Some risks reduced**, **other risks increased**
  - **Increased perceived value:** delivering some benefits early
  - **More flexibility**
  - **Better cost management**
- Each feature must be fully developed, to the extent it can be shipped.
- Develop features in order of ***priority***.

Done means Done

- Features in an iteration, must be 100% complete by the end of the iteration.
  - Ideally, each iteration results in a release.
- In Agile development, "DONE!" means shippable.
  - In practice a feature may rely on work in progress
    - But the feature on its own merit should be shippable.
- Completing each feature before moving to the next ensures system is not in a state where multiple features are 90% complete or untested



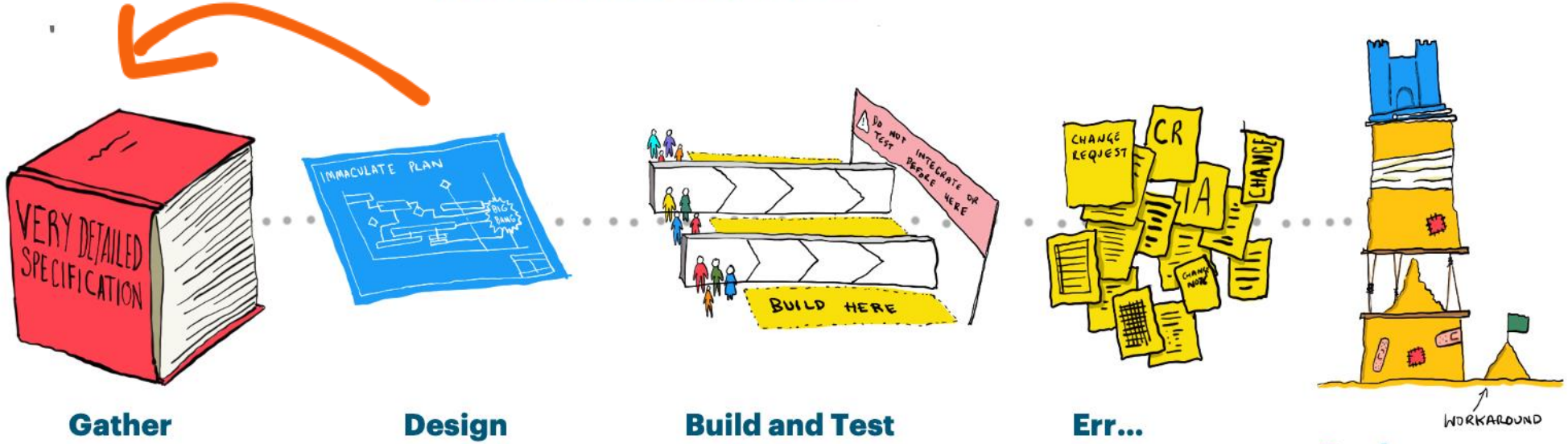
Working product at all times II



- The best way to get user feedback is to give a product even if it is only work in progress.
- Prototypes are better than a document.
- Effort spent getting the product back to a working state is a missed opportunity to be doing valuable work.

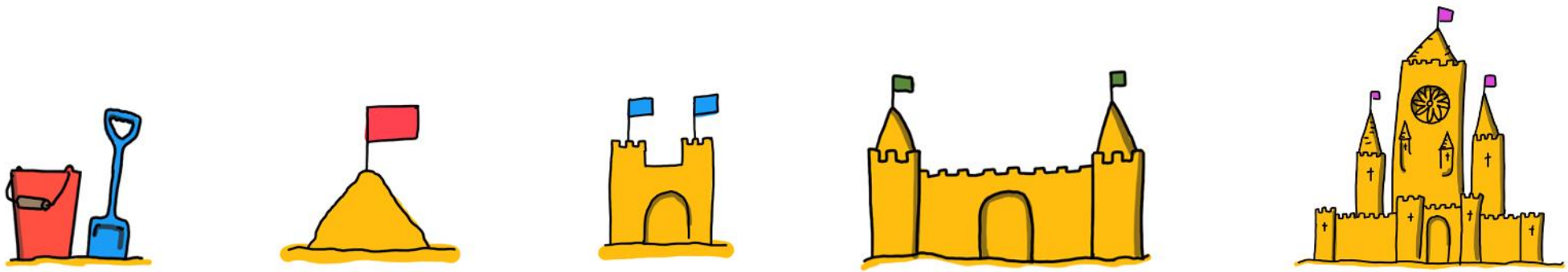
AGILE SANDCASTLES

WATERFALL



Gather Requirements → **Design** → **Build and Test** → **Err...** → **Deployment**

AGILE



Sprint 0 → **Sprint 1** → **Sprint 2** → **Sprint 3** → **Sprint 4**

accenture | SOLUTIONS IQ

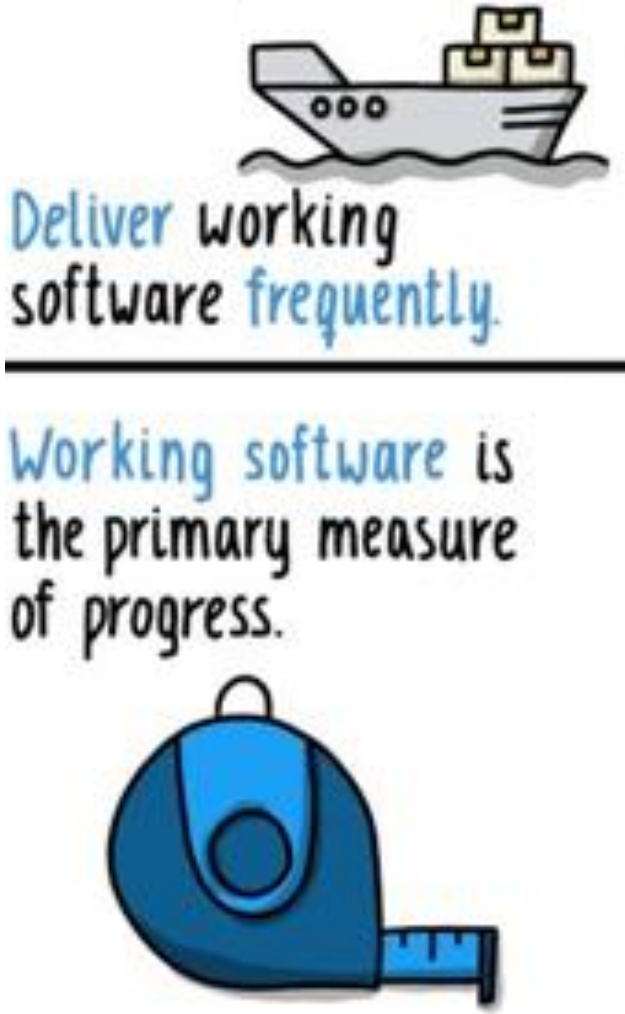
Designed by Kit Friend

Working Product at All Times – Prototypes

- Prototype solutions to *risky* problems helps to increase the chance of having a working product.
- Prototypes: an inexpensive way to try out ideas so that as many issues as possible are understood before the real implementation
- Two main classes of prototypes
  - the true prototype,
    - test implementation to understand a problem before it is implemented for real.
  - Skeleton
    - prototype that is intended to gradually turn into the final solution

Working product at all times – continuous integration

- Continuously integrate changes!
  - **Frequent integration helps modules fit together;**
  - product continues to work after changes.
- Developers bad habit:
  check out files, not check until after work is done.
  - integrate your work daily.
  - gradual introduction of changes ensures integration problems or regressions are caught early.



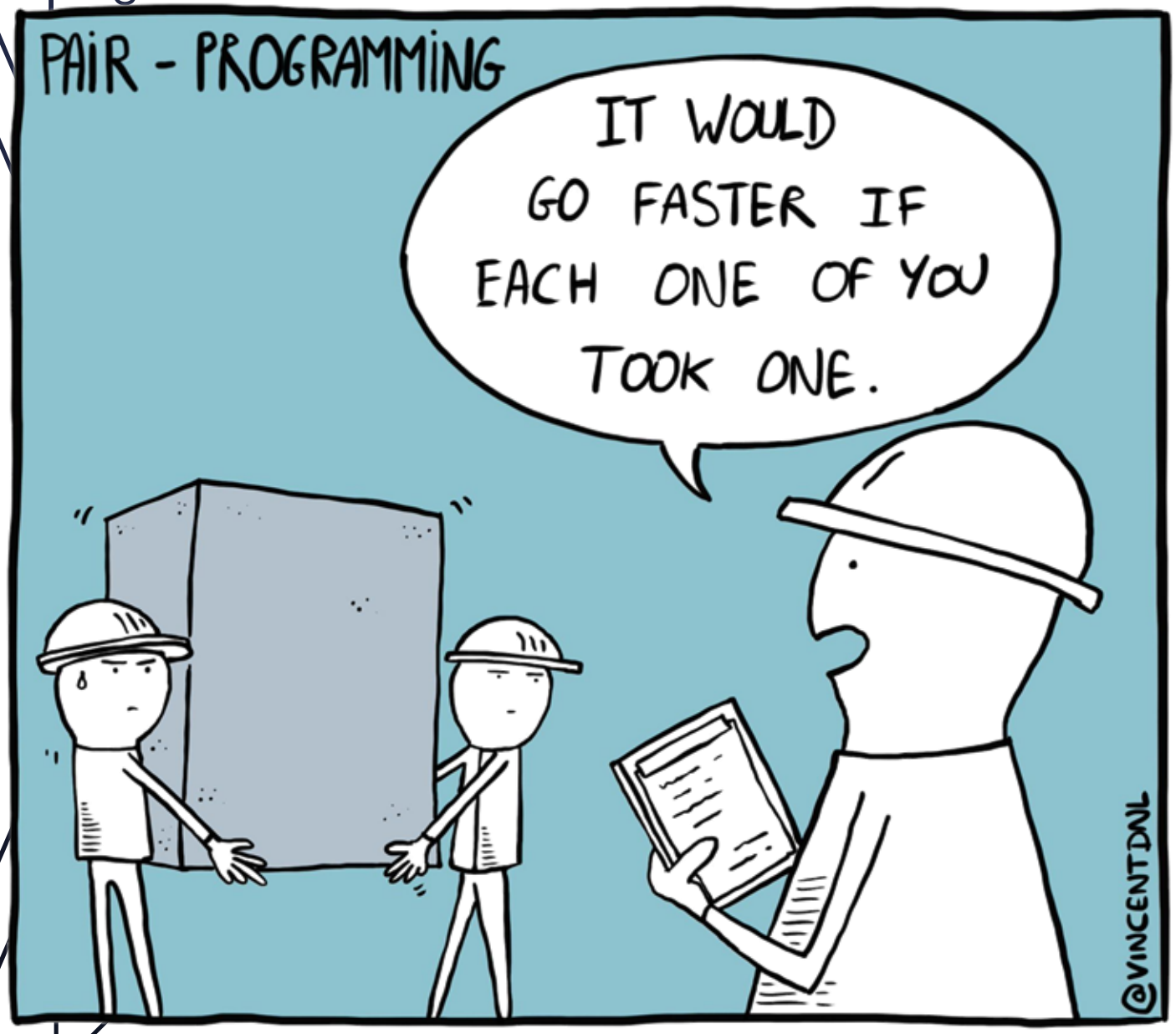
Working product at all times – nightly builds

- Software should be completely rebuilt from scratch daily.
  - the result of the build should be an installable product image.
- Build should include as many automated tests as possible to catch integration problems early.
  - If the build or tests fail, first fix problems
  - no-one integrates additional work until after the build succeeds again
  - otherwise: multiple bad changes accumulate risks the quality of the product.

Working product at all times – performance

- *Don't Neglect Performance*!
  - Performance is a topic that generates passionate discussions in software development.
  - Some people feel that code clarity is more important and that you should get the code clarity right first and then optimize the 1% to 3% of code that needs it.
  - Others feel that you should code for performance first, because if you don't, your code will always be slow.
  - A few critical loops dominate performance
  - Use profiling tools





CSC3003S ASD03
AGILE

A. Overview of Agile principles

B. Bare requirements

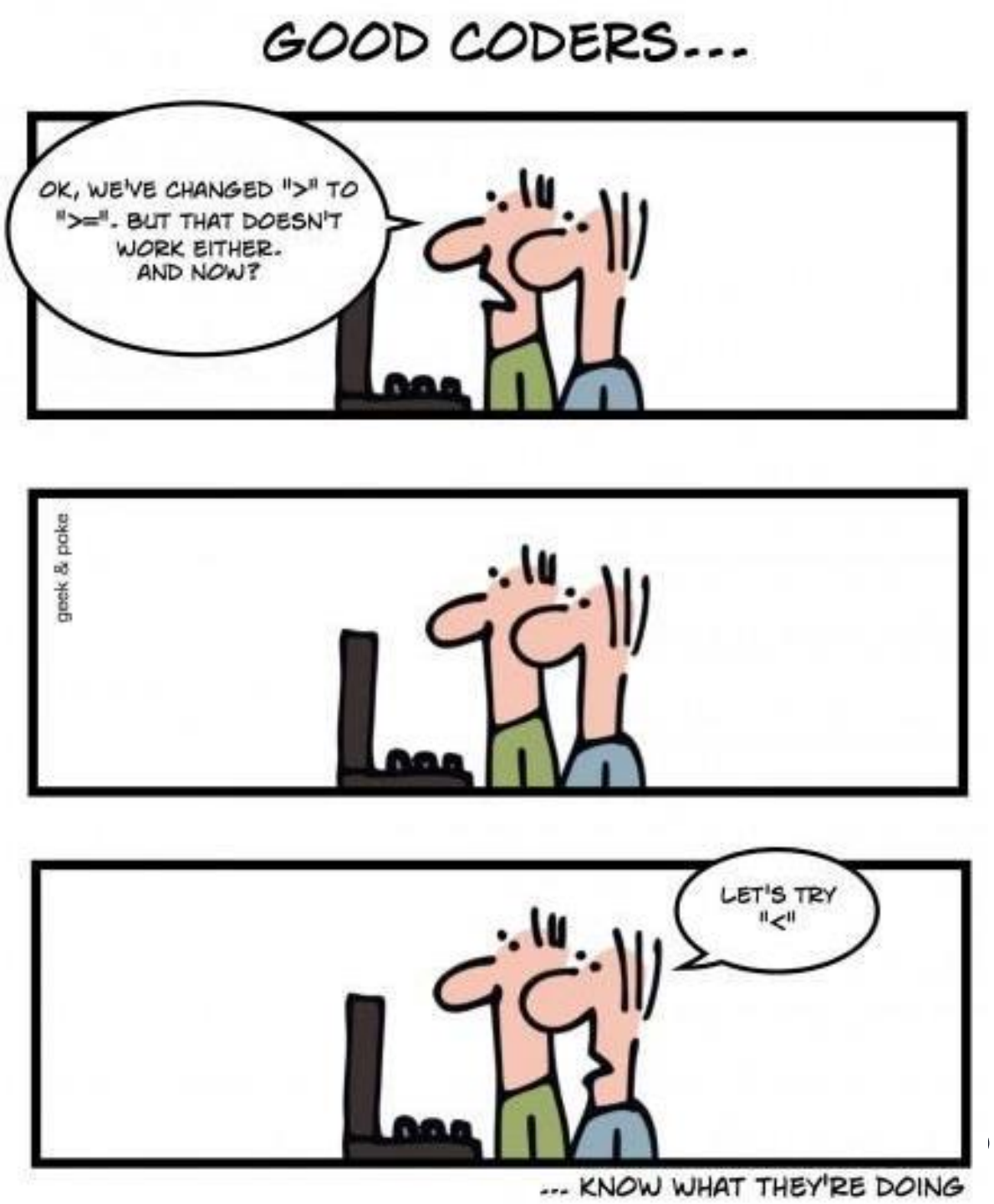
C. Fixed timescale

D. Working product at all times

E. Pair programming

Pair Programming

- Programmers work in pairs, sit together to write every line of code.
  - 2 programmers + 1 computer = atomic unit of XP code development.
  - One person at the keyboard, other supporting.
  - Pairs are created dynamically
  - Egoless development



Advantages of pair programming

- Informal review process:

  each line of code is looked at by at least two people.

- Common ownership of code
  - Individuals are not held responsible for problems with the code.
- Collective responsibility for the system.
  - Team has collective responsibility for resolving problems.
- Spreads knowledge across the team.
  - Reduces risk if someone leaves
- Motivates refactoring as the whole team will benefit from it.
  - Refactoring: improvement in quality of code without altering its functionality

Productivity and pair programming

- Productivity is similar to that of two people working independently …
  - Fewer false starts and less rework
  - Errors avoided due to informal inspection

    ➔ less time repairing bugs
- Pairing experienced with inexperienced developers
  - helps with training
- However: experienced developers may lose productivity, for some quality benefits
- Pair for complex tasks, maybe not for the simple ones.

Challenges of pair programming

- Working at partner's pace can be exhausting!
  - both too slow & too fast is tiring
- Constant communication requires empathy and interpersonal skills
- Different skill levels makes it difficult for pairs to estimate contributions and pace
- Power dynamics (e.g.: gender, seniority, race, degree)
- No time for yourself (introverts recharge during alone time)
- Pair rotations lead to context switching



Code review

- A good code review process also ensures that every change is reviewed by at least one other.
- The challenge is:
  - Easy to do sloppy or superficial reviews
  - Over-reliance of developer on code reviewer
    - little communication between programmer and reviewer
  - Sunk cost – code is already done "wasted effort" to tell programmer to refactor
  - Long time between making and finding bugs
    - *'what was I thinking when I wrote that code'*